Supplementary Material for CountTRuCoLa: Rule Learning for Interpretable Temporal Knowledge Graph Forecasting

Anonymous authors

No Institute Given

A Special remark on *c*-rules

In this section we provide a more detailed consideration of c-rules with the help of an example. Consider a dataset where we observed several times that persons, who eats pizza at timestamp t, will drink espresso at a subsequent timestamp. This regularity can be represented by the following c-rule:

$$drinks(x, espresso, t^\star) \leftarrow eats(x, pizza, t) \tag{1}$$

This c-rule can be applied to answer a query (Q1) $drinks(sally, ?, t^\star)$. Let us assume that Q1 has been derived from the triple $drinks(sally, espresso, t^\star)$. If we replace the subject instead of the object by a ?, this yields query (Q2') $drinks(?, espresso, t^\star)$. However, within our approach we do not support queries that ask for a subject directly. Instead, we translate them into equivalent object queries that use the inverse relation. We get (Q2) $drinks^{-1}(espresso, ?, t^\star)$.

Can we use the regularity expressed in Rule 1 to make a prediction for (Q2)? Obviously not directly, because this rule uses a different head relation. If write down Rule 1 with an inverse head, we will get the following rule:

$$drinks^{-1}(espresso, y, t^\star) \leftarrow eats^{-1}(pizza, y, t) \tag{2}$$

This rule is not supported by our language bias, because we do not support rules where the constant appears in subject position. This means that we are forced to resort to Rule 1 for answering (Q2') directly. The candidates which are predicted as answer to (Q2') are then combined with the answers for Q2 that we might get from other rules.

However, when selecting the training examples in Line 6 of Algorithm 1 we assumed that we have to deal with queries like (Q1) only. When asking a query as $drinks(sally, ?, t^\star)$ we can be sure that there is a correct candidate for the question mark and that is why we selected only those examples where, with respect to our example regularity, something has been drunk. If we focus on queries as (Q2) $drinks^{-1}(?, espresso, t^\star)$, we are in a different situation. We know already that Espresso has been drunk, and we ask who was drinking it. This means that for such a query we can limit our attention to training examples that fulfill the condition $\exists z\ drinks(z, espresso, t^\star) \in G$.

Thus, we have to distinguish between two application scenarios for c-rules and for these two application scenarios we have to collect different example sets associated to a c-rule. If we are concerned with object-queries, queries that have the question mark in the object position, we use Algorithm 1 and especially Line 6 as depicted in the main paper. This means that we check the condition $\exists z\ h(c, z, t^\star) \in G$. If, on the contrary, we are concerned with subject queries, queries that have the question mark in the subject position, we replace the condition in Line 6 in Algorithm 1 by $\exists z\ h(z, d, t^\star) \in G$ where d is the constant used in the head of the rule. As we distinguish between two examples sets associated to a c-rule, we also learn two confidence functions associated to each c-rule. Depending on the type of query, we apply one or the other confidence function to compute the confidence for a prediction. Within the code and the rule files we refer to these two variants as forward (F) and backward (B) rules respectively.

B Additional Information on Datasets

Table 1: Dataset statistics. # Quads refers to the number of quadruples without inverse quadruples.

Dataset	smallpedia	polecat	icews	wikidata	ICEWS14	ICEWS18	GDELT	YAGO	WIKI
# Quads Train	387,757	1,246,556	10,861,600	6,982,503	74,845	373,018	1,734,399	161,540	539,286
# Quads Valid	81,033	266,736	2,326,157	1,434,950	8,514	45,995	238,765	19,523	67,538
# Quads Test	81,586	266,318	2,325,689	1,438,750	7,371	49,545	305,241	20,026	63,110
# Nodes	47,433	150,931	87,856	1,226,440	7,128	23,033	7,691	10,623	12,554
# Relations	283	16	391	596	230	256	240	10	24
# Timestamps	125	1,826	10,224	2,025	365	303	2,975	188	231
Granularity	year	day	day	year	day	day	15 min	year	year

We provide additional details on the datasets used in our experiments. For each dataset, we use the version provided by [11] and [5], or, if lowercase letters only, the datasets introduced by [3]. An overview on the statistics of the datasets is in Table 1.

ICEWS Datasets: `ICEWS14` [2], `ICEWS18` [8], and `icews` [3] are derived from the Integrated Crisis Early Warning System (ICEWS) [1, 14]. These datasets span different periods (2014, 2018, and 1995–2022) and contain event data on global political activities such as conflicts, protests, and diplomatic interactions. Events are categorized according to the CAMEO taxonomy [6].

GDELT: The Global Database of Events, Language, and Tone (`GDELT`) [10] contains large-scale event data extracted from global news sources. It encompasses a wide range of political, societal, and cultural events across various countries and timeframes.

polecat: Based on the POLECAT (POLitical Event Classification, Attributes, and Types) dataset [13], this dataset records cooperative and hostile inter-

actions between socio-political actors. POLECAT uses the PLOVER ontology [7] and automated NLP pipelines to classify and extract time-stamped, geolocated events from multilingual news sources. The dataset used in this work covers the period from January 2018 to December 2022.

YAGO and WIKI: `YAGO` [12] and `WIKI` [9] provide structured knowledge graph data with temporal relations. `WIKI` is extracted from Wikidata [16] and both datasets have been further processed by [8] to represent temporal facts as quadruples. Events before 1786 (`WIKI`) and 1830 (`YAGO`) are excluded.

smallpedia and wikidata: These datasets are constructed from Wikidata [16] and processed by [3]. `smallpedia` includes entities with IDs below 1 million, while `wikidata` extends the scope to entities with IDs up to 32 million. Both datasets contain event-based (point-in-time) and fact-based (duration) temporal relations between entities.

C Details on Experimental Setup

C.1 Reasons for Excluding Prior Methods from Comparison

As noted in Section 5 in the main paper, we exclude 17 prior methods from direct comparison due to various limitations. In this section, we provide detailed justifications for each exclusion.

INFER reports result on the "best" ranking protocol in the presence of ties. As a result, the evaluation always assigns the best possible rank to the ground-truth entity in the case of ties, rather than using an average or random tie-breaking strategy. This protocol is known to inflate evaluation metrics unfairly, since it does not reflect the true ranking uncertainty in the presence of score ties [15]. Moreover, the provided code cannot be executed as it lacks the necessary specifications for pretraining the required Complex model. L2TKG, TPAR, Logenet, CRAFT, CoH, ALREIR and TECHS do not provide code to reproduce results. For HERLN, it is unclear under which filtering setting (raw, static, or time-aware) the reported results were obtained. The released code computes raw and time-aware scores, and the paper does not specify which setting is used in the main experiments. In addition, the official implementation cannot be executed due to missing files and unspecified configurations, resulting in runtime errors. We opened GitHub issues and contacted the authors for clarification, but did not receive a response. CENET, RETIA, and CluSTER do not report results in time-aware filter setting. CyGNet and RE-Net run only in multi-step setting, not in single-step setting. TR-Rules uses different dataset versions for ICEWS14 and does not report results on WIKI, YAGO, GDELT, or the TGB 2.0 datasets. GenTKG evaluates on different versions of GDELT and YAGO, does not report results on WIKI or the TGB 2.0 datasets, and does not provide MRR scores for any dataset. Finally, we do not compare to zrLLM because it uses a different evaluation setup, focusing on zero-shot relations with different datasets and on predicting previously unseen relations.

C.2 Notes on Experiments for TempValid and Dimnet

For both TempValid and DiMNet, we report results from our own experiments, conducted using the official public GitHub repositories[1]. In both cases, we carefully verified the evaluation protocols before running the experiments.

Despite following the provided code bases and reported settings, we encountered difficulties in reproducing the scores presented in the original papers. Table 2 reports our results and the reported ones. We could not determine the cause of these differences. One possible explanation are hyperparameter values that were not documented in the papers and repositories. Whenever hyperparameters were specified in the original papers, we used those values. Otherwise, we relied on the default settings from the released code. For datasets that were not included in the original works, we selected hyperparameters based on the datasets that were most similar in size and temporal granularity (`GDELT` for `polecat` and `icews`; `WIKI` for `smallpedia` and `wikidata`), since no hyperparameter-tuning scripts were provided.

Additionally, for TempValid, we encountered technical issues when running experiments on `polecat`, `icews`, and `wikidata`. Execution terminated with a runtime exception ("ValueError: Values are too large to be losslessly converted to uint16."). We suspect this is due to the larger scale of these datasets. The GitHub repository did not allow opening new issues. Furthermore, we observed out-of-memory (OM) errors in TempValid even when restricting execution to a single process as well as out-of-time (OT) errors.

Table 2: Results for TempValid and DiMNet on five benchmark datasets. Both papers did not report results on the datasest from TGB 2.0. OM means Out Of Memory (40 GB GPU or 500 GB RAM), - means that no results have been reported. The term orig refers to the results reported in the original papers, the term ours to the results from our experiments.

	GDELT		YAGO		WIKI		ICEWS14		ICEWS18	
	MRR	H10	MRR	H10	MRR	H10	MRR	H10	MRR	H10
TempValid orig	21.9	37.0	79.7	85.7	83.2	97.5	45.8	65.1	33.5	52.3
TempValid ours	OM	OM	46.7	50.4	OT	OT	**45.5**	**64.5**	OT	OT
DiMNet orig	21.9	37.5	-	-	-	-	45.7	68.0	34.1	55.8
DiMNet ours	21.1	36.5	72.4	82.7	71.2	79.7	41.4	60.8	33.9	55.4

C.3 Negative Samples

Following the TGB 2.0 evaluation framework, we use the provided negative samples for the large dataset `tkgl-wikidata`. Specifically, TGB 2.0 includes 1,000

[1] `https://github.com/Kaka1aaaaa/TempValid` for TempValid and `https://github.com/hhdo/DiMNet` for DiMNet

negative samples per query, sampled based on the query relation type. As a result, evaluation on `tkgl-wikidata` is performed by ranking the correct entity against these 1,000 candidates rather than against all entities. For all other datasets, we compute scores by ranking against the full set of entities.

D Hyperparameters

CountTRuCoLa comes with nine hyperparameters:

- $\mathcal{P}$: A constant added to the denominator when computing confidences for xy- and c-rules.
- $\mathcal{P}_{f\text{-rules}}$: The corresponding constant used for f-rules.
- $\mathcal{C}$: A boolean flag indicating whether c-rules are used for a given dataset.
- $\mathcal{W}$: The window size defining the length of the time window for the examples E, i.e., the maximum value in Δ.
- $\mathcal{M}$: A threshold defining the minimum number of data points required before learning the parameters of the frequency-based scoring function g_r. If a rule has fewer than $\mathcal{M}$ examples, all parameters of g_r are set to zero to reduce noise.
- $\mathcal{Z}$: A scaling factor $\mathcal{Z} \in [0, 1]$ applied to the score predicted by the z-rules.
- $\mathcal{H}$ and $\mathcal{D}$: Two hyperparameters for rule aggregation. $\mathcal{H}$ specifies how many of the top-confidence rules to aggregate, and $\mathcal{D} \in [0, 1]$ is a decay factor used in our modified noisy-or model, where the i-th confidence score s_i is weighted by $s_i \cdot \mathcal{D}^i$.
- $\mathcal{C}_{x\text{-}count}$: A threshold controlling the construction of c-rules. A rule candidate is retained only if its head and body relation–object pairs occur at least $\left\lfloor \frac{\mathcal{C}_{x\text{-}count}}{5} \right\rfloor$ times in the training data. We further evaluate rule support over all head quadruples matching the rule by sampling five past timestamps per quadruple. A rule is kept if the resulting groundings cover more than $\mathcal{C}_{x\text{-}count}$ distinct x-values in total.

We perform grid search on the validation MRR to select the values of these hyperparameters for each dataset. For smaller datasets, we explore a broader range of values, while for larger datasets, we reduce the search space to limit runtime and energy consumption. Table 3 lists the tested ranges for each dataset. Note that the range for $\mathcal{W}$ varies depending on the total number of timesteps available in each dataset. The value of $\mathcal{C}_{x\text{-}count}$ is set depending on dataset size: If a dataset has more than 10^6 quadruples, $\mathcal{C}_{x\text{-}count} = 20$, otherwise $\mathcal{C}_{x\text{-}count} = 3$. This provides a practical trade-off that enables scalable c-rule mining on large datasets, where exhaustive enumeration of all candidates is infeasible.

Table 4 reports the hyperparameter values selected for each dataset.

E Hyperparameter Sensitivity

Figure 1 shows the effect of selected hyperparameters on the test MRR across six datasets. We analyze the impact of $\mathcal{H}$ (`NUM_TOP_RULES`), $\mathcal{D}$ (`AGGREGATION_DECAY`),

Table 3: Hyperparameter ranges. We allow fewer values for larger datasets to reduce computation costs.

	small `WIKI`, `ICEWS14`, `YAGO`, `smallpedia`	medium `ICEWS18`	large `polecat`, `wikidata`, `GDELT`	very large `icews`
$\mathcal{P}$ (RULE_UNSEEN_NEGATIVES)	$\{0, 1, 2, 3, 5, 10, 20, 30, 100\}$	$\{1, 5, 10, 30, 100\}$	$\{1, 10, 30, 100\}$	$\{1, 10, 30, 100\}$
$\mathcal{P}_{f\text{-rules}}$ (F_UNSEEN_NEGATIVES)	$\{0, 1, 5, 10, 20, 30\}$	$\{0, 10, 30\}$	$\{10\}$	$\{10\}$
$\mathcal{C}$ (RULE_TYPE_C)	{True, False}	{True, False}	{True, False}	{False}
$\mathcal{W}$ (LEARN_WINDOW_SIZE)	$\{50, 100, 150\}$ `ICEWS14`, $\{2, 3, 5, 10, 30, 50, 100\}$ `WIKI`, $\{2, 3, 5, 10, 30, 50, 100\}$ `smallpedia`, $\{2, 3, 5, 10, 30, 30, 50\}$ `YAGO`	$\{50, 100, 150\}$	$\{50, 100, 150\}$ `polecat`, $\{2, 3, 5, 10, 30, 50\}$ `wikidata`, $\{10, 30, 50, 100, 150, 200\}$ `GDELT`	$\{50, 100\}$
$\mathcal{M}$ (DATAPOINT_THRESHOLD_MULTI)	$\{0, 10, 50\}$	$\{0, 50\}$	$\{0, 50\}$	$\{50\}$
$\mathcal{Z}$ (Z_RULES_FACTOR)	$\{0, 0.1, 0.2, 0.3, 0.4, 0.5, 1.0\}$	$\{0, 0.1, 0.5\}$	$\{0, 0.1, 0.5\}$	$\{0, 0.1, 0.5\}$
$\mathcal{H}$ (NUM_TOP_RULES)	$\{5, 10, 50\}$	$\{10, 50\}$	$\{10\}$	$\{10\}$
$\mathcal{D}$ (AGGREGATION_DECAY)	$\{1, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4\}$	$\{1, 0.8, 0.6\}$	$\{0.8\}$	$\{0.8\}$

Table 4: Hyperparameter values for each dataset, selected based on the validation MRR. The column `default` represents default hyperpararmeters that we represent, when no hyperparameter tuning can or should be conducted.

	`GDELT`	`YAGO`	`WIKI`	`ICEWS14`	`ICEWS18`	`smallp.`	`polecat`	`icews`	`wikidata`	`default`
$\mathcal{P}$ (RULE_UNSEEN_NEGATIVES)	30	30	1	30	100	3	30	100	30	30
$\mathcal{P}_{f\text{-rules}}$ (F_UNSEEN_NEGATIVES)	10	30	10	10	10	30	10	10	10	10
$\mathcal{C}$ (RULE_TYPE_C)	False	True	True	True	True	True	False	False	True	True
$\mathcal{W}$ (LEARN_WINDOW_SIZE)	200	30	3	50	50	3	150	50	10	10
$\mathcal{M}$ (DATAPOINT_THRESHOLD_MULTI)	0	50	50	0	50	10	0	50	0	0
$\mathcal{Z}$ (Z_RULES_FACTOR)	0.1	0.1	0.1	0.1	0.1	0	0.1	0.1	0.1	0.1
$\mathcal{H}$ (NUM_TOP_RULES)	10	5	10	10	10	5	10	10	10	10
$\mathcal{D}$ (AGGREGATION_DECAY)	0.8	0.7	0.8	0.8	0.8	0.4	0.8	0.8	0.8	0.8
$\mathcal{C}_{x\text{-}count}$ (C_X_COUNT)	20	3	3	3	3	3	20	-	20	3

and $\mathcal{W}$ (`LEARN_WINDOW_SIZE`). Note that the hyperparameters selected for CountTRuCoLa (Table 4) are based on validation performance and do not necessarily result in the highest test MRR.

Learn Window Size $\mathcal{W}$: Across all datasets, $\mathcal{W}$ has the largest effect on performance. For the Wikidata- and Yago-based datasets (`WIKI`, `YAGO`, `smallpedia`), the best results are achieved with a small window of 3, while the other datasets benefit from larger windows. This can likely be explained by two factors. First the Temporal granularity: Yearly datasets (`WIKI`, `YAGO`, `smallpedia`) cover multiple years even with small windows, whereas daily (`ICEWS14`, `ICEWS18`, `polecat`) or 15-min (`GDELT`) datasets require larger windows to capture sufficient history; Second, the Recurrency of facts: As observed by [4, 3], `WIKI`, `YAGO`, and `smallpedia` exhibit high direct recurrency ($> 85\%$), i.e., most temporal triples repeat in the previous timestep. Leveraging only recent history is sufficient, while incorporating older facts may introduce noise.

Number of Top Rules $\mathcal{H}$: For most datasets, performance improves with larger $\mathcal{H}$, with no substantial improvements between $\mathcal{H} = 10$ and $\mathcal{H} = 50$. Wikidata- and Yago-based datasets show a slight performance decrease as $\mathcal{H}$ increases, though the effect is minor. For a more detailed analysis of the effect of rule aggregation we refer to Section F.1.

Aggregation Decay $\mathcal{D}$: The aggregation decay $\mathcal{D}$ controls the assumed correlation between rules. $\mathcal{D} = 0$ corresponds to max-aggregation strategy, i.e., equals $\mathcal{H} = 1$, assuming highly correlated rules. In contrast, $\mathcal{D} = 1$ corresponds to noisy-or top-$\mathcal{H}$ aggregation, assuming independence between rules. For most datasets, intermediate values ($0.5 \leq \mathcal{D} \leq 0.8$) yield the best performance, reflecting partial correlation between rules. This aligns with the intuition that some rules are correlated, but not all. Wikidata-based datasets achieve the highest MRR with small $\mathcal{D}$ and $\mathcal{H}$, likely because predictions are dominated by recurrent rules, and additional correlated rules provide limited benefit.

Comparison to Default Hyperparameters Table 5 compares test MRR when hyperparameters are selected based on validation performance (as reported in Appendix D and Table 4) versus using default hyperparameter values, which are reported in the last column of Table 4 and were chosen based on intuition and general insights. The results indicate that hyperparameter selection generally improves performance. For example, the difference for `GDELT` ranges from 20.3 to 23.8, whereas other datasets, such as `polecat` and `YAGO`, are relatively stable, with differences of 0.6 and 0.1, respectively.

F Additional Results

F.1 Rule Aggregation Analysis

To study how aggregation affects the relationship between predicted confidence and empirical correctnes we analyze the correspondence between aggregated con-

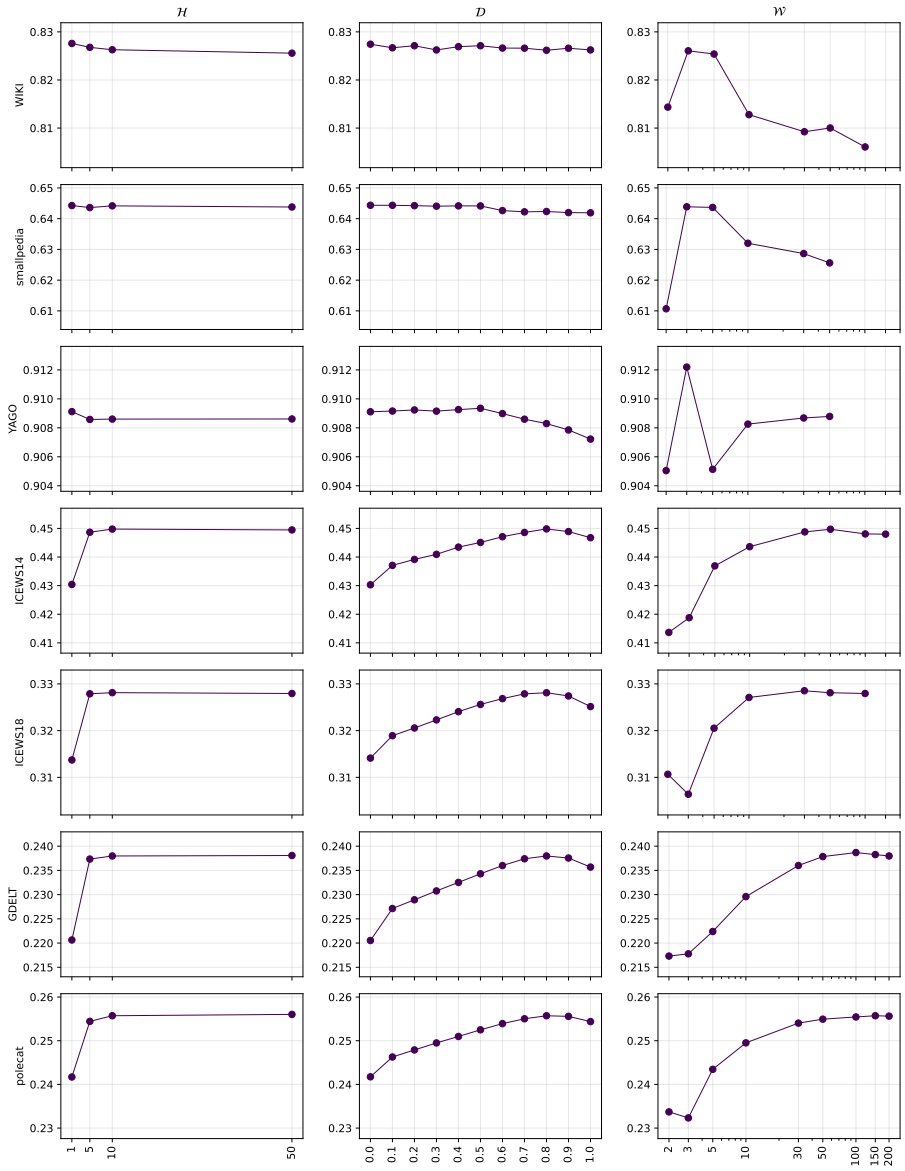
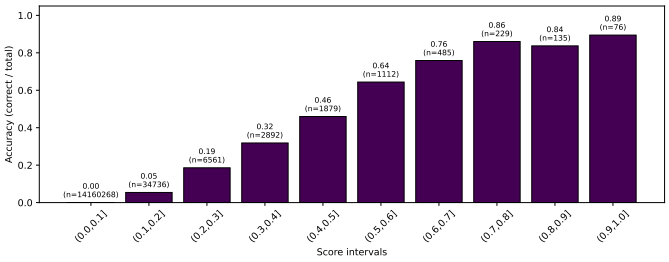
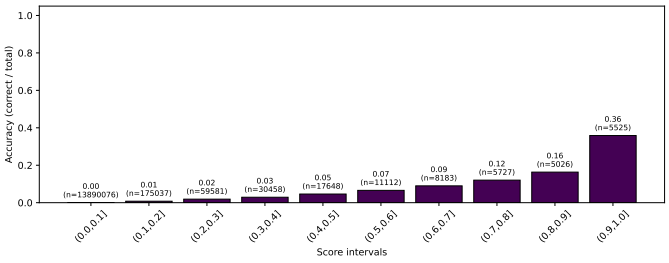


Fig. 1: Effect of the hyperparameters $\mathcal{H}$ (`NUM_TOP_RULES`), $\mathcal{D}$ (`AGGREGATION_DECAY`), and $\mathcal{W}$ (`LEARN_WINDOW_SIZE`) on the test MRR. Columns correspond to hyperparameters (left to right), and rows correspond to datasets.

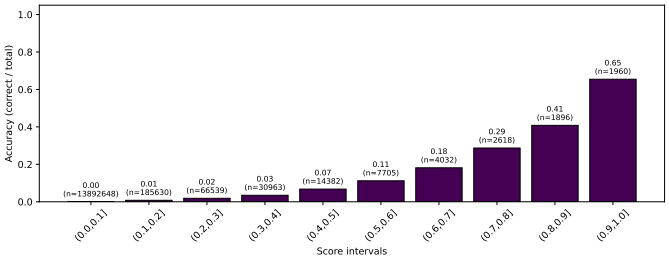
(a) $\mathcal{H} = 1$



(b) $\mathcal{H} = 100$ and $\mathcal{D} = 1$



(c) $\mathcal{H} = 10$ and $\mathcal{D} = 1$



(d) $\mathcal{H} = 10$ and $\mathcal{D} = 0.8$

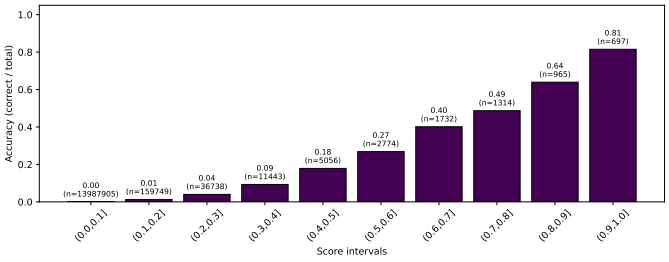


Fig. 2: Analysis of the relationship between aggregated rule confidence and empirical correctness for ICEWS14. Candidates are grouped into ten confidence bins, and the fraction of correct predictions in each bin is reported for four aggregation settings, where $\mathcal{H}$ represents `(NUM_TOP_RULES)`, and $\mathcal{D}$ `(AGGREGATION_DECAY)`.

Table 5: Test MRRs when selecting hyperparameters for each dataset (based on the validation MRR) vs. when using standard hyperparameter values.

	GDELT	YAGO	WIKI	ICEWS14	ICEWS18	smallp.	polecat	icews
selected per dataset	0.238	0.909	0.826	0.450	0.328	0.644	0.256	0.214
default hyperparams	0.203	0.908	0.812	0.444	0.312	0.630	0.250	0.208

fidence scores and ground-truth correctness under different aggregation configurations. This analysis allows us to study how the aggregation hyperparameter $\mathcal{H}$ (`NUM_TOP_RULES`) and the decay hyperparameter $\mathcal{D}$ (`AGGREGATION_DECAY`) influence the aggregated confidence values and the extent to which correlated rules may inflate them.

Analysis of Over- and Underestimation in Confidence Bins In this analysis, for each test query, we collected all predicted candidates together with their aggregated confidence scores and ground-truth labels (true or false). We then group the candidates in ten confidence bins $(i, i+0.1]$ with i in $0, 0.1, 0.2, \ldots, 0.9$ based on their assigned confidence score. For each bin, we computed the fraction of correct predictions.

The resulting curves indicate whether higher aggregated scores correspond to higher accuracy, thereby providing a diagnostic of whether CountTRuCoLa produces meaningful confidence values. We report results for four aggregation strategies: (a) max-aggregation ($\mathcal{H} = 1$), which uses only the highest-scoring rule and thus assumes strong dependence among rules; (b) top-100 noisy-or aggregation ($\mathcal{H} = 100$ and $\mathcal{D} = 1$), which combines the 100 highest-scoring rules under a stronger independence assumption; (c) top-10 noisy-or aggregation ($\mathcal{H} = 10$ and $\mathcal{D} = 1$), which combines the ten highest-scoring rules under a slightly weaker independence assumption; and (d) decayed noisy-or aggregation ($\mathcal{H} = 10$ and $\mathcal{D} = 0.8$) which reduces the influence of lower-ranked rules and thus aims to mitigate potential inflation from correlated rule sets.

Figure 2 reports the results for ICEWS14. The max-aggregation strategy (a) exhibits underestimation: Specifically, bins in the range 0.5–0.8 achieve higher empirical accuracies than their assigned confidence values, and relatively few candidates receive scores above 0.9. In contrast, the top-100 noisy-or strategy (b) shows substantial overestimation, with predicted scores exceeding accuracy across all bins and a substantially larger fraction of candidates receiving scores above 0.9. The top-10 noisy-or strategy (c) still overestimates confidence, but the inflation effect is noticeably reduced. Finally, the decayed noisy-or strategy (d) shows intermediate behavior. While a slight degree of overestimation remains, its predicted confidence values more closely match the empirical accuracies, and a monotonic relationship between confidence bins and accuracy is preserved.

Analysis of Over- and Underestimation in Top-1 Predictions To complement the analysis above, we further examine the confidence assigned to the top-ranked predictions. Table 6 reports, for different $(\mathcal{H}, \mathcal{D})$ settings, the achieved Hits@1

and the mean score assigned to the top-scoring candidate on `ICEWS14`. Ideally, the assigned score should align with the achieved Hits@1. Systematically higher scores indicate overestimation, for example due to overcounting correlated rules, while systematically lower scores indicate underestimation.

The results reflect the expected behavior. Max-aggregation underestimates confidence: with $\mathcal{H} = 1$ and $\mathcal{D} = 1$, the model assigns a lower score than the observed Hits@1, reflecting overly strong dependence assumptions. In contrast, a large $\mathcal{H} = 100$ leads to clear overestimation, with the predicted score far exceeding the Hits@1. This demonstrates that noisy-or indeed overcounts correlated rules when many rules are aggregated. A moderate setting of $\mathcal{H} = 10$ partially alleviates this effect, though still produces overconfident estimates. Introducing a decay factor ($\mathcal{H} = 10$, $\mathcal{D} = 0.8$) results in a closer match between predicted scores (42.2) and Hits@1 (35.9), while also producing the highest Hits@1 among the tested configurations.

Together with the trends in Figure 2, these results provide empirical evidence that, while not being able to fully hinder them, the decay parameter mitigates overcounting effects.

Table 6: Hits@1 and the mean score assigned to the top-scoring candidate for different $(\mathcal{H}, \mathcal{D})$ settings on `ICEWS14`.

$\mathcal{H}$	$\mathcal{D}$	Hits1	Mean Score
1	1	32.9	27.1
100	1	34.0	56.6
10	1	35.6	51.3
10	0.8	35.9	42.2

F.2 Analysis of Parameter Sensitivity to Learn Window Size

Figures 3 and 4 illustrate the impact of the Learn Window Size $\mathcal{W}$ on the distribution of learned parameters across the datasets `ICEWS14`, `GDELT`, and `WIKI`.

Recall that the recency function f is parameterized by λ, α, ϕ, which respectively control the exponential rate of decay, the starting value at $\Delta = 1$, and lower bound of the curve, while the frequency function g is parameterized by ρ, κ, and γ, controlling its slope, shift, and value clipping. Each rule learns its own set of these parameters.

Figure 3 summarizes the effect of $\mathcal{W}$ on the learned recency curves, showing for each dataset the mean curve together with its 10th and 90th percentile variation bands for three representative window sizes. The figure also includes violin plots depicting, across rules, the distributions of λ, α, and ϕ for different $\mathcal{W}$.

For the yearly dataset `WIKI`, the learned α values are consistently larger than in the other datasets across all window sizes, indicating that rules assign comparatively high confidence to events observed at $\Delta = 1$. The values of λ are

also generally higher, leading to steeper decay curves. This behavior stands in contrast to GDELT and ICEWS14, whose learned α values are substantially smaller, indicating lower confidence in general. Both of these datasets have much finer temporal granularity (15-minute and daily), so events are observed at a much higher density. In such settings, a single additional day or timestep carries less significance, and the model correspondingly learns shallower decay.

Despite differences in scale, GDELT and ICEWS14 exhibit similar trends as $\mathcal{W}$ increases. Larger windows tend to reduce both α and ϕ, while the smallest λ values appear at intermediate window sizes. The variance of λ grows with $\mathcal{W}$, suggesting that broader learning windows allow the rules to specialize more strongly. Some rules learn sharply decaying recency curves, whereas others maintain relevance more gradually over longer time spans, leading to a diverse collection of temporal behaviors.

Figure 4 presents the corresponding analysis for the frequency function. For ICEWS14 and GDELT, increasing $\mathcal{W}$ leads to frequency curves that become both steeper and higher. An interpretation is that with more historical context available, CountTRuCoLa places greater weight on frequency and learns to differentiate reliable, frequently recurring patterns from noisy or isolated events. At small $\mathcal{W}$, by contrast, frequency becomes less informative, which is reflected in flatter curves. At the extreme case of ICEWS14 with $\mathcal{W} = 3$, some rules even learn significantly negative ρ, indicating situations in which additional occurrences reduce the score, whic is likely an effect of noise or event sparsity in very short windows. Such behavior disappears at larger windows and is less occurent for the other datasets. WIKI shows a different pattern. Its frequency curves remain comparatively flat for all window sizes, and the distributions of ρ and κ show smaller variation.

Taken together, these observations show that CountTRuCoLa does not converge to a single temporal pattern. Instead, it learns a diverse set of curves, with substantial variation in parameters such as λ, ρ, and κ across rules. GDELT and ICEWS14 exhibit similar parameter behaviors across window sizes, whereas WIKI behaves fundamentally differently due to its yearly resolution and high direct recurrence.

Importantly, the results provide no evidence that recency or frequency parameters are transferable across datasets with different temporal granularities. Rather, the fitted parameters adapt strongly to the underlying time scale: coarser datasets learn sharper recency effects and flatter frequency responses, while finer-grained datasets support a wide spectrum of decay rates and frequency sensitivities. The choice of window size interacts closely with temporal resolution, and the learned parameters remain dataset-specific rather than universal.

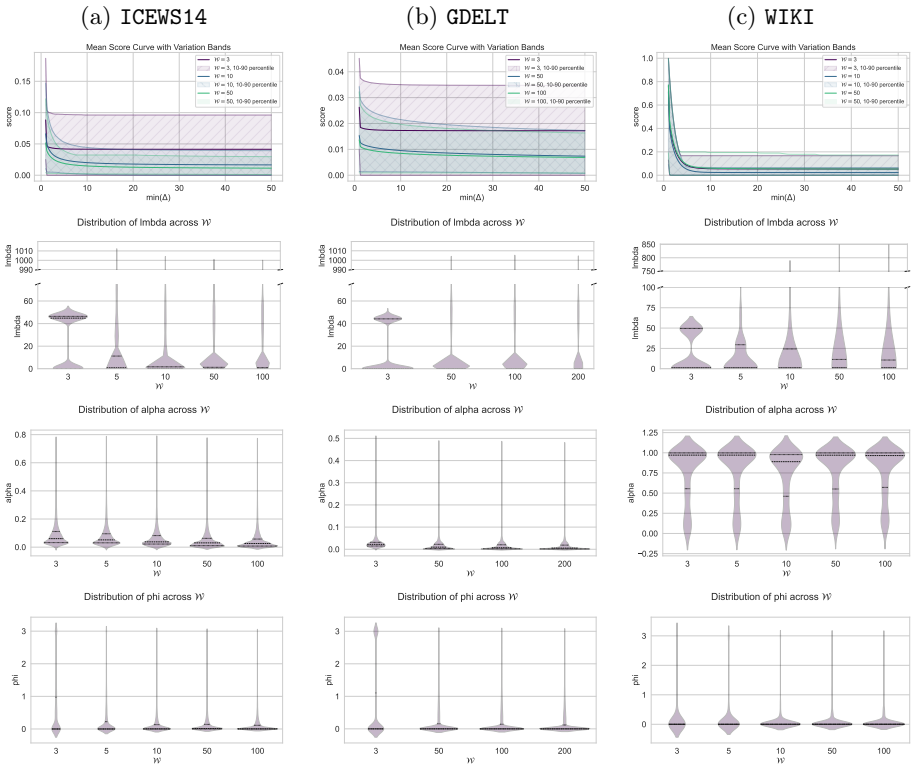


Fig. 3: Recency function f and the effect of Learn Window Size $\mathcal{W}$ on learned parameters for datasets `ICEWS14`, `GDELT`, and `WIKI` (left to right). Note that the y-axis varies across datasets.

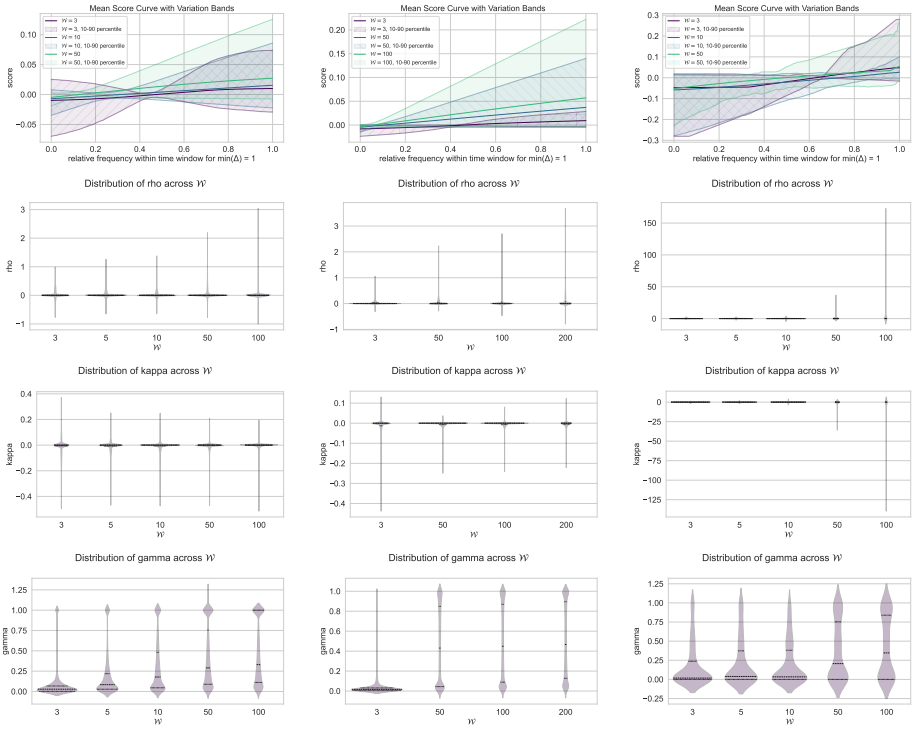


Fig. 4: Frequency function g and the effect of Learn Window Size $\mathcal{W}$ on learned parameters for datasets `ICEWS14`, `GDELT`, and `WIKI` (left to right). Note that the y-axis varies across datasets.

F.3 Rule Type Distributions

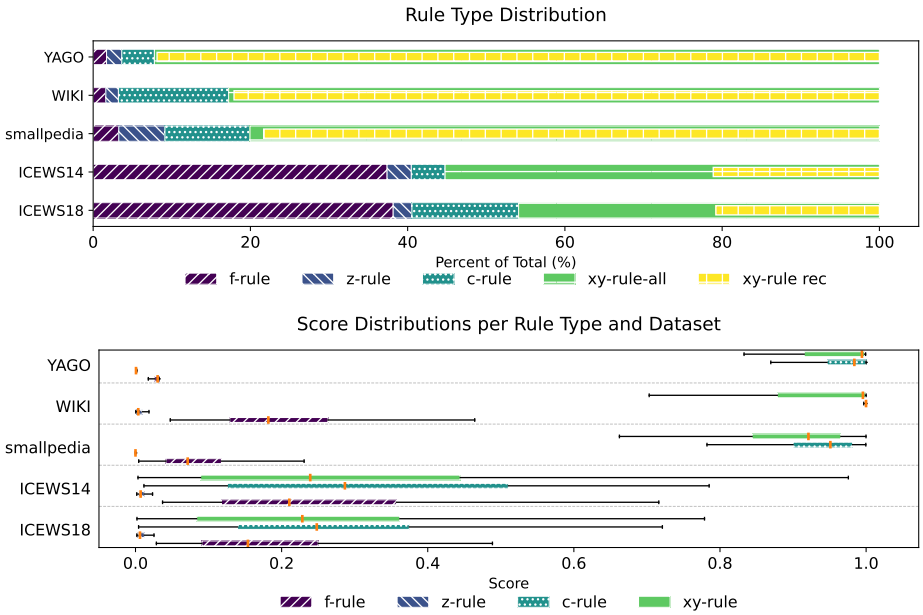


Fig. 5: Distributions for the four rule types (top), and assigned scores for each rule type (bottom) for the highest predicted node and highest predicting rule for each test query.

Figure 5 shows the distributions of rule types (top) and the corresponding scores (bottom) for the highest-predicted node and its top-predicting rule across all test queries in the small and medium sized datasets.

The figure illustrates that the contributions of rule types vary significantly across datasets. For datasets based on Wikidata (`WIKI`, `smallpedia`) and YAGO (`YAGO`), which exhibit a high degree of recurrency [4], xy-recurrency rules have the highest contributions. This is consistent with the ablation results in Table 2(a) (main paper), where CountTRuCoLa on `WIKI` achieves its best MRR using only recurrency rules. In contrast, for `ICEWS14` and `ICEWS18`, recurrency rules account for only around 20% of the contribution. Large contributions come also from non-recurrent xy-rules and f-rules, again reflecting the trends observed in Table 2(a) (main paper). Across all datasets, z-rules contribute minimally ($< 5\%$).

The score distributions in Figure 5 (bottom) show that xy- and c-rules achieve higher average scores for Wikidata- and YAGO-based datasets, whereas their average scores are substantially lower in `ICEWS14` and `ICEWS18`. The z-rules consistently have low median scores, as they primarily capture general distributions. The c-rules show the highest median scores for most datasets, which aligns with

their specialized nature and ability to provide targeted, high-confidence predictions.

Overall, these distributions confirm our intuition: xy-rules contribute the most across datasets, but other rule types also play an important role in achieving high performance.

F.4 Runtime

Table 7 reports runtimes in seconds for CountTRuCoLa on all datasets. We report the total runtime (including dataset loading and all necessary steps), as well as the runtimes of the individual components described in the main paper, i.e., creating the Examples E for each rule, learning the parameters for each rule, rule application and aggregation, and evaluation.

All runtimes were measured on an AMD EPYC 9474F 48-Core Processor running AlmaLinux 9.5. CountTRuCoLa does not require GPU acceleration and thus no GPUs were used in these experiments. Parallelization during rule application was implemented via Ray using up to 20 parallel workers.

For reference, Table 8 reports runtimes from [3] on the datasets included in their study. In addition, Table 9 reports runtimes for CognTKE on the datasets where we evaluated it, and Table 10 and Table 11 report runtimes for TempValid and DiMNet, respectively. Since these results were obtained on different hardware[2], they are not directly comparable, but they provide a useful point of reference.

Table 8 shows that CountTRuCoLa achieves substantially lower total runtimes on `smallpedia` and `polecat` than the runtimes reported for related work, despite running without GPU acceleration. On `icews`, CountTRuCoLa attains comparable runtimes, while on `wikidata` it is the only method aside from the simple EdgeBank heuristic that completes successfully. Table 9 further reports CognTKE runtimes on the datasets where we evaluated it. While these results were obtained on different hardware and are therefore not directly comparable, they provide an indication that CountTRuCoLa is highly efficient in practice, achieving low runtimes using only CPU resources.

F.5 Hits@1 and Hits@3 Results

Tables 12 and 13 report the results including Hits@1 and Hits@3 values.

The Hits@1 results do not provide a different impression from the MRR results presented in Table 1 (main paper). Whenever CountTRuCoLa ranks first, second, or third in MRR among the compared models, it has the same position with respect to the Hits@1 score. The only exceptions are `YAGO`, where

[2] Experiments in Table 8 were conducted on Nvidia A100, V100, V100SXM2, and RTX8000 GPUs with 4 CPU nodes (from AMD Rome, Milan, or Intel Skylake) per experiment, using up to 1056 GB of RAM, and experiments in Table 9, Table 10, and Table 11 were conducted on an NVIDIA RTX A6000 GPU with 48 GB VRAM with up to 16 CPU nodes, using up to 500 GB of RAM

Table 7: CountTRuCoLa runtimes for all datasets in seconds.

	GDELT	YAGO	WIKI	ICEWS14	ICEWS18	smallp.	polecat	icews	wikidata
# rules	134,574	8,140	350	53,553	1,010,307	1,221	1,024	167,670	12,050
creation of examples [s]	12,244	9	10	111	242	7	1,464	129,139	280
parameter learning [s]	5,670	11	0.4	366	7,743	1	594	2,221	18
rule application [s]	3,704	125	106	279	12,593	194	10,957	137,574	39,911
evaluation [s]	253	45	18	64	510	69	620	6335	2,637
total time [s]	21,926	191	143	825	21,149	280	13,681	276,269	43,029

Table 8: Inference time as well as total train and validation times as reported in [3] in seconds.

Method	smallpedia		polecat		icews		wikidata	
	Test	Total	Test	Total	Test	Total	Test	Total
$\text{EdgeBank}_{\text{tw}}$	2,935	5,810	46,629	94,475	311,278	600,929	5,445	8,875
RecurrencyBaseline	310	9,895	3,392	80,378	3,928	148,710	-	-
RE-GCN	165	3,895	1,766	45,877	6,848	114,370	-	-
CEN	331	14,493	2,726	77,953	8,999	202,477	-	-
TLogic	331	803	75,654	138,636	60,413	128,391	-	-

Table 9: Test time as well as total train and validation times for CognTKE in seconds.

	GDELT		YAGO		ICEWS14		smallp.		polecat		icews		wikidata	
	Test	Total	Test	Total	Test	Total	Test	Total	Test	Total	Test	Total	Test	Total
CognTKE	OM	OM	78	8,287	36	6,080	711	63,300	7,548	417,600	OM	OM	OM	OM

Table 10: Runtimes for TempValid in seconds.

	GDELT	YAGO	WIKI	ICEWS14	ICEWS18	smallp.	polecat	icews	wikidata
# rules	100,187	46	574	28,661	42,531	3,362	2,614	101,244	6,544
rule learning [s]	8,364	11	92	224	743	3,362	711	80,117	363,813
feature generation [s]	OM	18,651	OT	22,702	OT	OM	error	error	error
tempvalid training and testing [s]	OM	41	OT	4,275	OT	OM	error	error	error
total time [s]	OM	18,703	OT	27,201	OT	OM	error	error	error

Table 11: Runtimes for DiMNet in seconds.

	GDELT	YAGO	WIKI	ICEWS14	ICEWS18	smallp.	polecat	icews	wikidata
	Total	Total	Total	Total	Total	Total	Total	Total	Total
DiMNet	OM	914	2,062	1,300	7,033	2,045	OM	OM	OM

CountTRuCoLa moves from first to second place for Hits@1, and `ICEWS18`, where it moves from third to second place. The ranking of the Hits@3 values is also consistently within the top three positions among related work.

For the TGB2.0 datasets in Table 13, no Hits@1 and Hits@3 results were reported for related work. We thus compare our results only against CognTKE. On one, CognTKE achieves higher Hits@1 and Hits@3 scores, on another CountTRuCoLa achieves the highest results, and on two datasets, CognTKE did not produce results due to OM errors.

Table 12: Model comparison including Hits@1 and Hits@3 on five datasets. OM means Out Of Memory (40 GB GPU or 1056 GB RAM), OT means Out Of Time (7 days), - means that no results have been reported. Best results are shown in bold and underlined font, second-best bold, third-best underlined.

	GDELT				YAGO				WIKI				ICEWS14				ICEWS18			
	MRR	H1	H3	H10	MRR	H1	H3	H10	MRR	H1	H3	H10	MRR	H1	H3	H10	MRR	H1	H3	H10
TRKG	21.5	13.7	24.0	37.3	71.5	65.7	77.3	79.2	73.4	71.2	75.6	76.2	27.3	16.5	31.1	50.8	16.7	8.3	18.2	35.4
xERTE	18.9	12.7	21.1	32.0	87.3	84.2	90.3	91.2	74.5	70.3	78.6	80.1	40.9	33.0	45.5	57.1	29.2	20.9	33.5	46.3
TANGO	19.2	12.2	20.4	32.8	62.4	59.0	64.7	67.8	50.1	48.3	51.4	52.8	36.8	27.3	40.8	55.1	28.4	19.1	31.9	46.3
Timetraveler	20.2	14.1	22.2	31.2	87.7	84.6	90.9	91.2	78.7	75.2	82.0	83.1	40.8	31.9	45.4	57.6	29.1	21.3	32.5	43.9
TiRGN	21.7	13.6	23.3	37.6	88.0	84.3	91.4	92.9	81.7	77.8	85.1	**87.1**	44.0	33.8	49.0	**63.8**	**33.7**	23.2	**38.0**	**54.2**
TempValid	OM	OM	OM	OM	46.7	43.3	50.0	50.4	OT	OT	OT	OT	**45.5**	35.4	**50.8**	**64.5**	OT	OT	OT	OT
DiMNet	21.1	13.3	22.5	36.5	72.4	67.1	75.0	82.7	71.2	66.4	74.0	79.7	41.4	31.4	46.3	60.8	33.9	23.2	38.2	55.4
CognTKE	OM	OM	OM	OM	90.6	88.1	**92.9**	**93.2**	**83.2**	**80.0**	**86.0**	**87.3**	**46.1**	**36.5**	**51.1**	**64.5**	**35.2**	**25.2**	**39.9**	**54.7**
TLogic	19.8	12.2	21.7	35.6	76.5	74.0	78.9	79.2	82.3	78.6	**86.0**	87.0	42.5	33.2	47.6	60.3	29.6	20.4	33.6	48.1
RE-GCN	19.8	12.5	21.0	33.9	82.2	78.7	84.2	88.5	78.7	74.8	81.7	84.7	42.1	31.4	47.3	62.7	32.6	22.4	36.8	52.6
CEN	20.4	13.0	21.8	35.0	82.7	78.8	85.2	89.4	79.3	75.5	82.4	84.9	41.8	31.9	46.6	60.9	31.5	21.7	35.4	50.7
EdgeBank$_{tw}$	1.9	0.1	0.9	3.5	61.7	44.1	68.5	61.7	58.5	39.4	67.3	84.4	13.5	3.2	14.1	34.2	7.2	1.5	6.3	17.9
Rec.B ($\psi_{\Delta}\xi$)	**24.5**	**16.4**	**26.8**	**39.8**	**90.9**	**89.0**	92.8	93.0	81.4	76.9	85.7	**87.1**	37.4	29.9	41.2	51.5	28.7	20.8	32.3	43.6
CountTRuCoLa	**23.8**	**15.4**	**26.3**	**40.3**	**90.9**	**88.8**	**92.9**	**93.2**	**82.7**	**79.4**	85.7	86.6	**45.0**	**36.0**	49.8	62.0	32.8	**23.5**	36.9	51.0

Table 13: Model comparison including Hits@1 and Hits@3 on the TGB2.0 benchmark datasets. OM means Out Of Memory (40 GB GPU or 1056 GB RAM), OT means Out Of Time (7 days), - means that no results have been reported. Best results are shown in bold and underlined font, second-best bold, third-best underlined.

	smallp				polecat				icews				wikidata			
	MRR	H1	H3	H10	MRR	H1	H3	H10	MRR	H1	H3	H10	MRR	H1	H3	H10
TempValid	OM	OM	OM	OM	error	error	error	error	error	error	error	error	error	error	error	error
DiMNet	54.3	**48.6**	57.4	64.7	OM	OM	OM	OM	OM	OM	OM	OM	OM	OM	OM	OM
CognTKE	53.4	44.0	**60.2**	70.1	**28.7**	**20.1**	**32.7**	**45.5**	OM	OM	OM	OM	OM	OM	OM	OM
TLogic	59.5	-	-	70.7	22.8	-	-	37.8	18.6	-	-	30.1	OT	OT	OT	OT
RE-GCN	59.4	-	-	68.7	17.5	-	-	29.2	18.2	-	-	**33.1**	OM	OM	OM	OM
CEN	**61.2**	-	-	70.5	18.4	-	-	32.3	18.7	-	-	**33.4**	OM	OM	OM	OM
EdgeBank$_{tw}$	35.3	-	-	56.6	5.6	-	-	11.9	2.0	-	-	5.8	**53.5**	-	-	**59.6**
Rec.B ($\psi_{\Delta}\xi$)	60.5	-	-	**71.6**	19.8	-	-	31.7	**21.1**	-	-	32.4	OT	OT	OT	OT
CountTRuCoLa	**64.4**	**59.5**	**68.7**	**71.7**	**25.6**	**17.8**	**29.1**	**40.8**	**21.4**	**15.7**	**24.1**	32.1	**60.9**	**59.8**	**61.6**	**62.8**

F.6 Variance Across Repetitions

Table 14 reports test results across five repetitions for two selected datasets. We observe that the variance is less than 0.002 for all test metrics and datasets, which is small considering that the MRR and Hits@10 range from 0% to 100%. Compared to embedding-based methods, which may introduce randomness through factors such as random initialization or noise injection, our approach has fewer potential sources of variability: (i) sampling candidates for the examples of c-rules, (ii) computing the parameters that optimize the temporal confidence functions, and (iii) assigning random order to tied ranks during evaluation.

Table 14: Test results on `WIKI` and `ICEWS14` (5 runs, mean, variance).

	`WIKI`		`ICEWS14`	
	MRR	H10	MRR	H10
	82.6809	86.5592	44.9916	62.0201
	82.6615	86.5568	44.9828	62.0404
	82.5704	86.5647	45.0014	62.1083
	82.6181	86.5624	44.9864	62.0336
	82.5992	86.5584	44.9685	62.0201
Mean	82.6260	86.5603	44.9862	62.0445
Variance	0.002038	0.000010	0.000146	0.001348

F.7 Significance Tests

We conduct a Chi-squared test to assess whether the observed differences in Hits@10 scores between methods are statistically significant. Since Hits@10 is a binary metric (a prediction is either in the top 10 or not), we treat the predictions as categorical outcomes: *hit* and *miss*. For a dataset, we have $2 \times N$ samples (where N is the number of test quadruples), accounting for inverse relations.

For example, if a model achieves a Hits@10 score of 0.6 on a test set with 100 quadruples, this corresponds to $0.6 \times 200 = 120$ *hit* predictions and 80 *miss* ones.

We compare CountTRuCoLa with the best-performing state of the art method regarding Hits@10 under the following null hypothesis:

> **$\mathbf{H_0}$**: Prediction correctness (i.e., *hit* vs. *miss*) is independent of the method used.

For examples, for the `GDELT` dataset, the Chi-squared test indicates a significant association between prediction correctness and method ($\chi^2 = 31.8$, $df = 1$, $p < 0.001$), suggesting that the improvement in Hits@10 by CountTRuCoLa over the baseline is statistically significant. Based on $p < 0.001$, in total, for the

4 datasets, where CountTRuCoLa has higher Hits@10 scores than the others, for 2 of them the Chi-squared tests suggests a statistically significant improvement. For the 5 datasets, where CountTRuCoLa has lower Hits@10 scores than the other methods, for 4 of them the Chi-squared test suggests a statistically significant difference.

Table 15: Chi-squared test for all datasets. We test for ($df = 1$, $p < 0.001$). and compare CountTRuCoLa with the best performing state of the art method regarding Hits@10.

Dataset	CountTRuCoLa H10	Best Baseline H10	Best Baseline	χ^2	Significant?
Group 1: CountTRuCoLa best method					
`GDELT`	40.3	39.8	Rec. B.	31.78	yes
`YAGO`	93.2	93.2	CognTKE	0	no
`smallpedia`	71.7	71.6	Rec. B.	0.40	no
`wikidata`	62.8	59.6	Edgebank	6204.43	yes
Group 2: CountTRuCoLa not best method					
`WIKI`	86.6	87.3	CognTKE	27.25	yes
`ICEWS14`	62.0	64.5	CognTKE	19.81	no
`ICEWS18`	51.0	54.7	CognTKE	272.19	yes
`icews`	32.1	33.4	CEN	1784.58	yes
`polecat`	40.8	45.4	CognTKE	2297.88	yes

F.8 Memory Limits

The upper memory limits set in SLURM for each dataset (using 20 parallel processes) are as follows:

- Small datasets (`WIKI`, `ICEWS14`, `YAGO`, `tkgl-smallpedia`): 20 GB
- `polecat`: 160 GB
- `ICEWS18`: 300 GB
- `gdelt`: 350 GB
- `wikidata`: 200 GB
- `tkgl-icews`: 500 GB

Please note that these are upper limits, not actual memory usage. Additionally, reducing the number of parallel processes during the application phase will decrease memory consumption at the cost of increased runtime.

G Details on the explanation tool

In the following, we provide additional details on the explanation tool introduced in Section 6 in the main paper.The code for the explanation tool is also published in the provided repository, along with notebooks demonstrating its use.

Pre-Analysis Features Before generating explanations, the tool helps gain an overview of datasets and models. It provides (i) dataset statistics like relation distributions, and number of triples over time, computes (ii) a fine-grained evaluation with metrics like MRR per relation and timestep, and conducts (iii) a ranking comparison that identifies quadruples where two methods differ significantly.

For example, the ranking comparison showed that on the dataset ICEWS14, CountTRuCoLa consistently outperforms RE-GCN on the relation *Consult*, suggesting that simple temporal rules capture this relation comparatively well.

Input and Workflow The tool takes as input (i) a rule set (learned or user-defined), (ii) an optional configuration (e.g., aggregation function, window size), and (iii) queries to explain. It then applies rules with CountTRuCoLa, records applied rules and scores, conducts an evaluation and generates visual explanations.

For instance, selecting all *Consult* queries where CountTRuCoLa outperforms RE-GCN enables targeted investigation. This allows researchers to inspect which rule patterns explain correct forecasts and where neural models may miss such dependencies.

Output For each query, the tool produces a structured explanation that includes the predicted node and score, the rules that contributed (with their parameters), visualizations of temporal confidence functions, and the frequency/recency of supporting quadruples. In addition, the tool outputs evaluation scores (MRR, Hits@k) for the quadruples of interest, allowing fine-grained comparison of settings.

H Examples for Learned Recency Functions

As explained in Section 4.2 in the main paper, every parameter in CountTRuCoLa has a fixed and interpretable role. Figure 6 illustrates the role of each parameter λ, α, ϕ in the recency function f for an individual rule. Figure 7 provides three concrete examples of rules that have learned different parameters, along with the corresponding sets of examples E that were used to learn these curves. Together, the figures highlight that different rules indeed need different parameters to describe their behaviour.

For example, the first rule exhibits a steep decay, with its confidence approaching zero as $\min(\Delta)$ increases. In contrast, the second rule requires a relatively shallow decay. The third rule does not converge to zero and is supported by positive examples at higher $\min(\Delta)$.

These observations align with our Ablation Study in Section 5.2, Table 2(c) in the main paper, which demonstrates that learning rule-specific parameters improves performance. In summary, the study confirms that different rules indeed require different confidence-function shapes, and modeling them individually is beneficial.

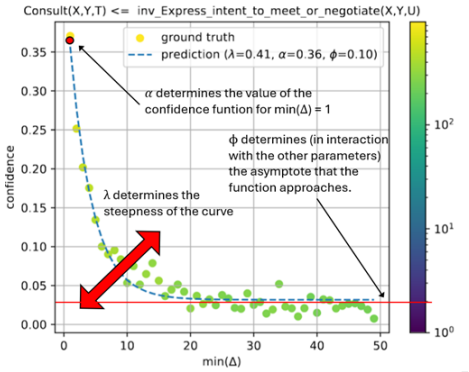


Fig. 6: Examples E (points) and predicted confidence curves (blue lines) f for a rule r. Colors indicate the number of samples in E. The text explains the role of each parameter λ, α, ϕ.

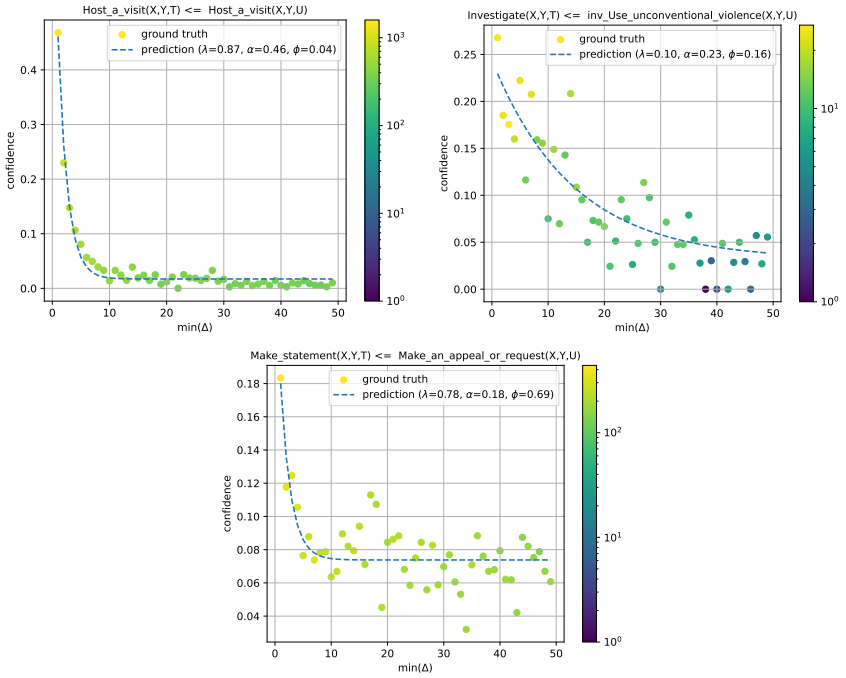


Fig. 7: Examples E (points) and predicted confidence curves (blue lines) f for three rules. Colors indicate the number of samples in E.

I Rule Length Study for TLogic

Table 16 shows test MRRs of TLogic with body length restricted to 1 ($\text{TLogic}_{\texttt{one}}$), TLogic with its original body lengths ($\text{TLogic}_{\texttt{orig}}$)[3], and CountTRuCoLa. Multi-hop rules yield noticeable gains on ICEWS14/18, while the remaining datasets even show marginal performance decrease. This indicates that restricting rule bodys to length 1 does not substantially reduce performance on many standard datasets. Consequently, extending CountTRuCoLa with multi-hop rules may yield additional gains mainly on datasets such as ICEWS14/18. However, this would come at the cost of increased runtime and reduced simplicity.

Table 16: Rule Body length study, test MRRs.

Dataset	ICEWS14	ICEWS18	YAGO	WIKI	GDELT
$\text{TLogic}_{\texttt{one}}$	40.7	27.5	76.4	82.4	20.0
$\text{TLogic}_{\texttt{orig}}$	42.5	29.6	76.5	82.3	19.8
CountTRuCoLa	45.0	32.8	90.9	82.7	23.8

[3] $\text{TLogic}_{\texttt{orig}}$ was tested on rules up to length 3 for ICEWS14/18 and YAGO, and up to length 2 for GDELT/WIKI. We omit other datasets where $\text{TLogic}_{\texttt{orig}}$ was tested only on length-1 rules.

References

1. Boschee, E., Lautenschlager, J., O'Brien, S., Shellman, S., Starz, J., Ward, M.: ICEWS Coded Event Data (2015)
2. García-Durán, A., Dumančić, S., Niepert, M.: Learning sequence encoders for temporal knowledge graph completion. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. pp. 4816–4821. Association for Computational Linguistics, Brussels, Belgium (Oct-Nov 2018)
3. Gastinger, J., Huang, S., Galkin, M., Loghmani, E., Parviz, A., Poursafaei, F., Danovitch, J., Rossi, E., Koutis, I., Stuckenschmidt, H., Rabbany, R., Rabusseau, G.: TGB 2.0: A benchmark for learning on temporal knowledge graphs and heterogeneous graphs. In: 38th Conference on Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track (2024)
4. Gastinger, J., Meilicke, C., Errica, F., Sztyler, T., Schuelke, A., Stuckenschmidt, H.: History repeats itself: A baseline for temporal knowledge graph forecasting. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI) (2024)
5. Gastinger, J., Sztyler, T., Sharma, L., Schuelke, A., Stuckenschmidt, H.: Comparing apples and oranges? On the evaluation of methods for temporal knowledge graph forecasting. In: Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML PKDD). pp. 533–549 (2023)
6. Gerner, D.J., Schrodt, P.A., Yilmaz, O., Abu-Jabr, R.: Conflict and mediation event observations (cameo): A new event data framework for the analysis of foreign policy interactions. International Studies Association, New Orleans (2002)
7. Halterman, A., Bagozzi, B.E., Beger, A., Schrodt, P., Scraborough, G.: Plover and polecat: A new political event ontology and dataset. In: International Studies Association Conference Paper (2023)
8. Jin, W., Qu, M., Jin, X., Ren, X.: Recurrent event network: Autoregressive structure inference over temporal knowledge graphs. arXiv preprint arXiv:1904.05530 (2019), preprint version
9. Leblay, J., Chekol, M.W.: Deriving validity time in knowledge graph. In: Champin, P., Gandon, F., Lalmas, M., Ipeirotis, P.G. (eds.) Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23-27, 2018. pp. 1771–1776. ACM (2018)
10. Leetaru, K., Schrodt, P.A.: Gdelt: Global data on events, location, and tone, 1979–2012. In: ISA annual convention. pp. 1–49. Citeseer (2013)
11. Li, Z., Jin, X., Li, W., Guan, S., Guo, J., Shen, H., Wang, Y., Cheng, X.: Temporal knowledge graph reasoning based on evolutional representation learning. In: The 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR) (2021)
12. Mahdisoltani, F., Biega, J.A., Suchanek, F.M.: Yago3: A knowledge base from multilingual wikipedias. In: CIDR (2015)
13. Scarborough, G.I., Bagozzi, B.E., Beger, A., Berrie, J., Halterman, A., Schrodt, P.A., Spivey, J.: POLECAT Weekly Data (2023)
14. Shilliday, A., Lautenschlager, J., et al.: Data for a worldwide icews and ongoing research. Advances in Design for Cross-Cultural Activities **455** (2012)
15. Sun, Z., Vashishth, S., Sanyal, S., Talukdar, P.P., Yang, Y.: A re-evaluation of knowledge graph completion methods. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 5516–5522 (2020)
16. Vrandečić, D., Krötzsch, M.: Wikidata: a free collaborative knowledgebase. Communications of the ACM **57**(10), 78–85 (2014)